# Lab 12

**Objective:** Learn about pointers to routines

***This lab builds on lectures 8, 9, 10, 11, 12***

**Introduction**

Pointers to routines in C are used in places where lambdas (or functors) are used in Java, C++, or other object oriented languages. For example, as the comparison operator when sorting, or as a thread start-up routine. Geeks4Geeks has a good tutorial "Function Pointer in C" (https://www.geeksforgeeks.org/function-pointer-in-c/) , and this tutorial is based on theirs.

**Exercise 1: Declare, assign to, and use a pointer to a routine.**

They start with an example where they declare a routine that just prints its integer parameter:

```
#include <stdio.h>
void fun(int_t a)
{
printf("Value of a is %d\n", a);
}
```

They then write a `main()` that demonstrates how to declare a pointer to a routine, assign to it, and how to use it:

```
int main()
{
void (*fun_ptr)(int) = fun;

fun_ptr(10);

return 0;
}
```

Enter this program the run it. Convince yourself that you understand how it works.

**Exercise 2: Array of pointers to routines**

In a separate program example, they show how an array of pointers to functions can make a program smaller, more efficient, and (once mastered) easier to write. They start with a number of routines that might get called. Notice how the declaration of each function (the *prototype*) of each function is the same:

```
#include <stdio.h>
void add(int a, int b)
{
printf("Addition is %d\n", a+b);
}

void subtract(int a, int b)
{
printf("Subtraction is %d\n", a-b);
}

void multiply(int a, int b)
{
printf("Multiplication is %d\n", a*b);
}
```

They then supply a `main()` that declares and initialises an array of pointers to routines. They then declare `a` and `b`, two operands to a user-selectable operation. Finally they ask the user which operation to perform and call the appropriate method through the function pointer in the array.

```
int main()
{
// fun_ptr_arr is an array of function pointers
void (*fun_ptr_arr[])(int, int) = {add, subtract, multiply};

unsigned int ch,
unsigned int a = 15
unsigned int b = 10;

printf("Enter Choice: 0 for add, 1 for subtract and 2 for multiply\n");
scanf("%d", &ch);

if (ch > 2)
  return 0;

(*fun_ptr_arr[ch])(a, b);

return 0;
}
```

Enter this program the run it. Convince yourself that you understand how it works.

**Exercise 3: `qsort()` integers**

A frequent use of pointers to routines in C is the comparison function to `qsort()` the built in quicksort method, `qsort()`, takes a pointer to a comparison function with this prototype:

```
int compare(const void *a, const void *b)
```

Here `void *` means "a pointer to an unspecified type". And `const` means we can't change what is being pointed to. So we must cast to get to the type we want. I strongly encourage you to be explicit in this as it makes debugging easier. We now declare a comparison routine that takes two integers and compares them

```
int compare (const void *a, const void * b)
{
int first = *(int*)a;
int second = *(int *) b;

if (first < second)
  return -1;
else if (first == second)
  return 0;
else
  return 1;
}
```

To call `qsort()` we need the array to sort, the number of elements in the array, the `sizeof()` each element in the array, and a pointer to a comparison routine. Geeks4Geeks gives this program:

```
int main ()
{
int arr[] = {10, 5, 15, 12, 90, 80};

qsort (arr, 6, sizeof(int), compare);

for (i = 0; i < n; i++)
  printf ("%d ", arr[i]);

return 0;
}
```

Enter this program the run it. Convince yourself that you understand how it works.

**Exercise 4: Sort an array of strings**

Write a program that uses `scanf()` to read 10 strings from the user, then sort those strings, then write them out in alphabetical order. To do this you might want to keep an array of pointers to strings and sort that array. To "keep" the strings you might want to use `malloc()` and `strcpy()`.

**Exercise 5: `bsearch()` an array of strings**

Other standard-library functions in C use pointers to routines. `bsearch()` has a prototype remarkably similar to `qsort()`. This is so that you can use the same comparison routine in both. `qsort()` then `bsearch()` is an very powerful and frequently used combination in C. The prototype for `bsearch()` is:

```
void *bsearch(const void *key, const void *base, size_t nitems, size_t
size, int (*compar)(const void *, const void *));
```

Extend your answer to exercise 4 so that the user can, after entering the strings, enter more strings and your program outputs "Y" or "N", whether or not those strings are in the initial list.